As well as literal text, route templates can include two other types of elements: parameters and separators, as shown in figure 4.3. Parameters are placeholders for dynamic values, like parameters for C# methods, and separators represent the boundary between segments in a URL. The difference between a literal and a parameter is that the latter is enclosed in curly braces. Parameters are an extremely powerful tool in terms of URL-to-endpoint matching. You will learn a lot more about working with parameters in routes soon, but before that, we will take a look at using literal text to override the default route template for a page.

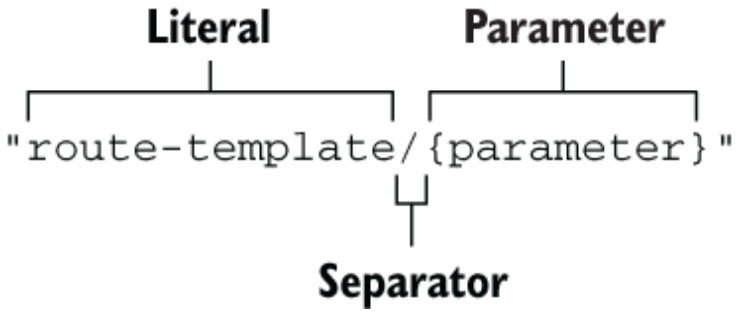


Figure 4.3 Demonstrates a literal within a route template, a separator character, and a placeholder for a parameter value within curly braces

4.2.1 Overriding routes

Suppose you find yourself in the kind of pickle illustrated in table 4.3, where two pages generate the same route template. In that case, you might want to override the template generated by one of them to prevent exceptions from being thrown at run time. You can do this by providing an alternative literal text value to the `@page` directive that starts with the separator character (`/`). For example, you can add `"/privacy-policy"` after the `@page` directive in the \Pages\Privacy.cshtml file like this:

```
@page "/privacy-policy"
```